珠海市杰理科技股份有限公司
ZhuHai JieLi Technology Co.,Ltd

获取 a2dp sbc 数据流及 sbc 解码文档

获取 a2dp sbc 数据流及 sbc 解码文档

一、获取 a2dp sbc 数据流

1、首先检查 audio_dec_bt.c 文件中是否有定义 audio_dec_input 结构体，并在 a2dp_dec_start 中有调用，如果有，则可以直接在 audio_dec_input 结构中的 audio_dec_input.ops.frame.get() 函数末尾，可以取到 sbc 数据流的源数据。

2、如果文件中没有定义 audio_dec_input 结构体，则需要通过以下两张图的方式，将 audio_dec_input 结构体先抽取出来，再重定义一个新的 audio_dec_input.ops.frame.get()函数封装一层，在原先操作后添加自定义的取数流程(最好把数据取走存放到另外的数据结构中，然后在其他线程处理，不然会堵塞解码线程)。

```
static int a2dp_dec_start(void)
{
    int err;
    struct audio_fmt *fmt;
    struct a2dp_dec_hdl *dec = bt_a2dp_dec;
    u8 ch_num = 0;
    u8 ch_type = 0;

    if (!bt_a2dp_dec) {
        return -EINVAL;
    }

    log_i("a2dp_dec_start: in\n");

    // 打开a2dp解码
    err = a2dp_decoder_open(&dec->dec, &decode_task);
    if (err) {
        goto __err1;
    }
    // 使能事件回调
    audio_decoder_set_event_handler(&dec->dec.decoder, a2dp_dec_event_handler, 0);

    save_input = dec->dec.decoder.input;
    save_decoder = &dec->dec.decoder;
    dec->dec.decoder.input = &tmp_input;

    // 获取解码格式
    err = audio_decoder_get_fmt(&dec->dec.decoder, &fmt);
    if (err) {
        goto __err2;
    }
```

```
////////////////////////////////////////////////////////////////
static struct audio_dec_input *save_input;
static struct audio_decoder *save_decoder;

static int tmp_dec_get_frame(struct audio_decoder *decoder, u8 **frame)
{
    int len = save_input->ops.frame.fget(save_decoder, frame);
    if (len) {
        spi_w(*frame, len);
    }
    return len;
}
static void tmp_dec_put_frame(struct audio_decoder *decoder, u8 *frame)
{
    save_input->ops.frame.fput(save_decoder, frame);
}
static int tmp_dec_fetch_frame(struct audio_decoder *decoder, u8 **frame)
{
    int len = save_input->ops.frame.ffetch(save_decoder, frame);
    return len;
}

static const struct audio_dec_input tmp_input = {
    .coding_type = AUDIO_CODING_SBC,
    .data_type   = AUDIO_INPUT_FRAME,
    .ops = {
        .frame = {
            .fget = tmp_dec_get_frame,
            .fput = tmp_dec_put_frame,
            .ffetch = tmp_dec_fetch_frame,
        }
    }
};
```

自定义的操作

二、自定义的 sbc 解码流程

详细代码在附录 demo_dec_frame_play.c 中，下面是调用解码的实际操作流程。

1、参考 demo_frame_test_time_func 函数，把需要解码的 sbc 的数据帧写入帧链表，后面解码流程开始后会主动从这个链表中按先进先出的顺序读取 sbc 数据帧（需要判断是否需要开始解码了，不然有可能会提前存下不必要的数据，导致后面需要的数据写不入，解码流程开始后会有数据断点）。

2、开始解码的时候调用 demo_frame_dec_open 函数开始解码流程。

3、解码线程会自动调用 demo_frame_dec_get_frame 函数获取 sbc 帧数据并进行解码，并按数据流顺序输出。

4、结束解码的时候调用 demo_frame_dec_close 函数结束解码流程。

附录：

demo_dec_frame_play.c

```
#include "asm/includes.h"
#include "system/includes.h"
#include "app_config.h"
#include "audio_config.h"
#include "audio_dec.h"
#include "clock_cfg.h"

struct demo_frame_decoder {
    u8 start;               // 解码开始
    u8 wait_resume;         // 需要激活
    int coding_type;    // 解码类型
    struct audio_decoder decoder;    // 解码器
    struct audio_res_wait wait;      // 资源等待句柄
    struct audio_mixer_ch mix_ch;    // 叠加句柄
    struct audio_stream *stream;     // 音频流
};

struct demo_frame_decoder *demo_frame_dec = NULL;

int demo_frame_media_get_packet(u8 **frame);
void demo_frame_media_free_packet(void *_packet);
void *demo_frame_media_fetch_packet(int *len, void *prev_packet);
int demo_frame_media_get_packet_num(void);

int demo_frame_dec_close(void);



// 解码获取数据
```

```
static int demo_frame_dec_get_frame(struct audio_decoder *decoder, u8 **frame)
{
    struct demo_frame_decoder *dec = container_of(decoder, struct demo_frame_decoder, decoder);
    u8 *packet = NULL;
    int len = 0;

    // 获取数据
    len = demo_frame_media_get_packet(&packet);
    if (len < 0) {
        // 失败
        putchar('X');
        return len;
    }

    *frame = packet;

    return len;
}

// 解码释放数据空间
static void demo_frame_dec_put_frame(struct audio_decoder *decoder, u8 *frame)
{
    struct demo_frame_decoder *dec = container_of(decoder, struct demo_frame_decoder, decoder);

    if (frame) {
        demo_frame_media_free_packet((void *)(frame));
    }
}

// 解码查询数据
```

```
static int demo_frame_dec_fetch_frame(struct audio_decoder *decoder, u8 **frame)
{
    struct demo_frame_decoder *dec = container_of(decoder, struct demo_frame_decoder, decoder);
    u8 *packet = NULL;
    int len = 0;
    u32 wait_timeout = 0;

    if (!dec->start) {
        wait_timeout = jiffies + msecs_to_jiffies(500);
    }

__retry_fetch:
    packet = demo_frame_media_fetch_packet(&len, NULL);
    if (packet) {
        *frame = packet;
    } else if (!dec->start) {
        // 解码启动前获取数据来做格式信息获取等
        if (time_before(jiffies, wait_timeout)) {
            os_time_dly(1);
            goto __retry_fetch;
        }
    }

    return len;
}

static const struct audio_dec_input demo_frame_input = {
    .coding_type = AUDIO_CODING_SBC,
    .data_type      = AUDIO_INPUT_FRAME,
    .ops = {
        .frame = {
```

```
                .fget = demo_frame_dec_get_frame,
                .fput = demo_frame_dec_put_frame,
                .ffetch = demo_frame_dec_fetch_frame,
            }
        }
};


// 解码预处理
static int demo_frame_dec_probe_handler(struct audio_decoder *decoder)
{
    struct demo_frame_decoder *dec = container_of(decoder, struct demo_frame_decoder, decoder);

    if (demo_frame_media_get_packet_num() < 1) {
        // 没有数据时返回负数，等有数据时激活解码
        dec->wait_resume = 1;
        audio_decoder_suspend(decoder);
        return -EINVAL;
    }
    return 0;
}

// 解码后处理
static int demo_frame_dec_post_handler(struct audio_decoder *decoder)
{
    return 0;
}
static const struct audio_dec_handler demo_frame_dec_handler = {
    .dec_probe  = demo_frame_dec_probe_handler,
    .dec_post   = demo_frame_dec_post_handler,
};
```

```
// 解码释放
static void demo_frame_dec_release(void)
{
    // 删除解码资源等待
    audio_decoder_task_del_wait(&decode_task, &demo_frame_dec->wait);

    clock_remove(DEC_SBC_CLK);

    // 释放空间
    local_irq_disable();
    free(demo_frame_dec);
    demo_frame_dec = NULL;
    local_irq_enable();
}

// 解码关闭
static void demo_frame_audio_res_close(void)
{
    if (demo_frame_dec->start == 0) {
        printf("demo_frame_dec->start == 0");
        return ;
    }

    // 关闭数据流节点
    demo_frame_dec->start = 0;
    audio_decoder_close(&demo_frame_dec->decoder);
    audio_mixer_ch_close(&demo_frame_dec->mix_ch);

    // 先关闭各个节点，最后才 close 数据流
    if (demo_frame_dec->stream) {
```

```
        audio_stream_close(demo_frame_dec->stream);
        demo_frame_dec->stream = NULL;
    }

    app_audio_state_exit(APP_AUDIO_STATE_MUSIC);
}

// 解码事件处理
static void demo_frame_dec_event_handler(struct audio_decoder *decoder, int argc, int *argv)
{
    switch (argv[0]) {
    case AUDIO_DEC_EVENT_END:
        printf("AUDIO_DEC_EVENT_END\n");
        demo_frame_dec_close();
        break;
    }
}

// 解码数据流激活
static void demo_frame_out_stream_resume(void *p)
{
    struct demo_frame_decoder *dec = (struct demo_frame_decoder *)p;

    audio_decoder_resume(&dec->decoder);
}

// 收到数据后的处理
void demo_frame_media_rx_notice_to_decode(void)
{
    if (demo_frame_dec && demo_frame_dec->start && demo_frame_dec->wait_resume) {
        demo_frame_dec->wait_resume = 0;
        audio_decoder_resume(&demo_frame_dec->decoder);
```

```
    }
}

// 解码 start
static int demo_frame_dec_start(void)
{
    int err;
    struct audio_fmt *fmt;
    struct demo_frame_decoder *dec = demo_frame_dec;

    if (!demo_frame_dec) {
        return -EINVAL;
    }

    printf("demo_frame_dec_start: in\n");

    // 打开 demo_frame 解码
    err = audio_decoder_open(&dec->decoder, &demo_frame_input, &decode_task);
    if (err) {
        goto __err1;
    }

    // 设置运行句柄
    audio_decoder_set_handler(&dec->decoder, &demo_frame_dec_handler);

    if (dec->coding_type != demo_frame_input.coding_type) {
        struct audio_fmt f = {0};
        f.coding_type = dec->coding_type;
        err = audio_decoder_set_fmt(&dec->decoder, &f);
        if (err) {
            goto __err2;
        }
```

```
}

// 获取解码格式
err = audio_decoder_get_fmt(&dec->decoder, &fmt);
if (err) {
    goto __err2;
}

// 使能事件回调
audio_decoder_set_event_handler(&dec->decoder, demo_frame_dec_event_handler, 0);

// 设置输出声道类型
audio_decoder_set_output_channel(&dec->decoder, audio_output_channel_type());

// 配置 mixer 通道参数
audio_mixer_ch_open_head(&dec->mix_ch, &mixer); // 挂载到 mixer 最前面
audio_mixer_ch_set_src(&dec->mix_ch, 1, 0);
audio_mixer_ch_set_no_wait(&dec->mix_ch, 1, 20); // 超时自动丢数
/* audio_mixer_ch_set_sample_rate(&dec->mix_ch, fmt->sample_rate); */

// 数据流串联
struct audio_stream_entry *entries[8] = {NULL};
u8 entry_cnt = 0;
entries[entry_cnt++] = &dec->decoder.entry;
// 添加自定义数据流节点等
// 最后输出到 mix 数据流节点
entries[entry_cnt++] = &dec->mix_ch.entry;

// 创建数据流，把所有节点连接起来
dec->stream = audio_stream_open(dec, demo_frame_out_stream_resume);
audio_stream_add_list(dec->stream, entries, entry_cnt);
```

```
    // 设置音频输出类型
    audio_output_set_start_volume(APP_AUDIO_STATE_MUSIC);

    // 开始解码
    dec->start = 1;
    err = audio_decoder_start(&dec->decoder);
    if (err) {
        goto __err3;
    }
    clock_set_cur();

    return 0;

__err3:
    dec->start = 0;

    audio_mixer_ch_close(&dec->mix_ch);

    // 先关闭各个节点，最后才 close 数据流
    if (dec->stream) {
        audio_stream_close(dec->stream);
        dec->stream = NULL;
    }

__err2:
    audio_decoder_close(&dec->decoder);
__err1:
    demo_frame_dec_release();

    return err;
}
```

```
// 解码资源等待回调
static int demo_frame_wait_res_handler(struct audio_res_wait *wait, int event)
{
    int err = 0;

    y_printf("demo_frame_wait_res_handler: %d\n", event);

    if (event == AUDIO_RES_GET) {
        // 可以开始解码
        err = demo_frame_dec_start();
    } else if (event == AUDIO_RES_PUT) {
        // 被打断
        if (demo_frame_dec->start) {
            demo_frame_audio_res_close();
        }
    }
    return err;
}

// 打开解码
int demo_frame_dec_open(void)
{
    struct demo_frame_decoder *dec;

    if (demo_frame_dec) {
        return 0;
    }

    printf("demo_frame_dec_open \n");

    dec = zalloc(sizeof(*dec));
    ASSERT(dec);
```

```
    clock_add(DEC_SBC_CLK);

    demo_frame_dec = dec;
    dec->coding_type = AUDIO_CODING_SBC;   // 解码类型
    dec->wait.priority = 1;          // 解码优先级
    dec->wait.preemption = 0;    // 不使能直接抢断解码
    dec->wait.snatch_same_prio = 1;  // 可抢断同优先级解码
    dec->wait.handler = demo_frame_wait_res_handler;
    audio_decoder_task_add_wait(&decode_task, &dec->wait);

    return 0;
}

// 关闭解码
int demo_frame_dec_close(void)
{
    if (!demo_frame_dec) {
        return 0;
    }

    if (demo_frame_dec->start) {
        demo_frame_audio_res_close();
    }
    demo_frame_dec_release();
    clock_set_cur();
    printf("demo_frame_dec_close: exit\n");
    return 1;
}

////////////////////////////////////////////////////////////////////////////////
```

```
//


struct demo_frame_media_rx_bulk {
    struct list_head entry;
    int data_len;
    u8 data[0];
};


static u32 *demo_frame_media_buf = NULL;
static LIST_HEAD(demo_frame_media_head);
u16 demo_frame_test_tmr = 0;
static u32 demo_frame_cnt = 0;
static u32 demo_frame_max = 0;


#define DEMO_FRAME_TIME     10


#define SBC_DATA_POINTS         32
#define SBC_DATA_SAMPRATE       44100
static const u8 sbc_data[119] = {
    0x9C, 0xBD, 0x35, 0x35, 0xF8, 0xE3, 0x22, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0xB9, 0xB8,
0xFE,
    0xC9, 0x3C, 0x18, 0xAE, 0xB4, 0x12, 0x2C, 0x8A, 0xC0, 0xC5, 0x51, 0x91, 0x80, 0x0D, 0x70,
0xB9,
    0x2A, 0x52, 0xF9, 0x72, 0xDB, 0x4D, 0xC9, 0x5B, 0x9B, 0x8F, 0xEC, 0x93, 0xC1, 0x8A, 0xEB,
0x41,
    0x22, 0xC8, 0xAC, 0x0C, 0x55, 0x19, 0x18, 0x00, 0xD7, 0x0B, 0x92, 0xA5, 0x2F, 0x97, 0x2D,
0xB4,
    0xDC, 0x95, 0xB9, 0xB8, 0xFE, 0xC9, 0x3C, 0x18, 0xAE, 0xB4, 0x12, 0x2C, 0x8A, 0xC0, 0xC5,
0x51,
    0x91, 0x80, 0x0D, 0x70, 0xB9, 0x2A, 0x52, 0xF9, 0x72, 0xDB, 0x4D, 0xC9, 0x5B, 0x9B, 0x8F,
0xEC,
    0x93, 0xC1, 0x8A, 0xEB, 0x41, 0x22, 0xC8, 0xAC, 0x0C, 0x55, 0x19, 0x18, 0x00, 0xD7, 0x0B,
```



```
0x92,
    0xA5, 0x2F, 0x97, 0x2D, 0xB4, 0xDC, 0x95
};



// 获取 frame 数据
int demo_frame_media_get_packet(u8 **frame)
{
    struct demo_frame_media_rx_bulk *p;
    local_irq_disable();
    if (demo_frame_media_head.next != &demo_frame_media_head) {
        p = list_entry((&demo_frame_media_head)->next, typeof(*p), entry);
        __list_del_entry(&p->entry);
        *frame = p->data;
        local_irq_enable();
        return p->data_len;
    }
    local_irq_enable();

    return 0;
}

// 释放 frame 数据
void demo_frame_media_free_packet(void *data)
{
    struct demo_frame_media_rx_bulk *rx = container_of(data, struct
demo_frame_media_rx_bulk, data);

    local_irq_disable();
    __list_del_entry(&rx->entry);
    local_irq_enable();
```

```
    lbuf_free(rx);
}

// 检查 frame 数据
void *demo_frame_media_fetch_packet(int *len, void *prev_packet)
{
    struct demo_frame_media_rx_bulk *p;
    local_irq_disable();
    if (demo_frame_media_head.next != &demo_frame_media_head) {
        if (prev_packet) {
            p = container_of(prev_packet, struct demo_frame_media_rx_bulk, data);
            if (p->entry.next != &demo_frame_media_head) {
                p = list_entry(p->entry.next, typeof(*p), entry);
                *len = p->data_len;
                local_irq_enable();
                return p->data;
            }
        } else {
            p = list_entry((&demo_frame_media_head)->next, typeof(*p), entry);
            *len = p->data_len;
            local_irq_enable();
            return p->data;
        }
    }
    local_irq_enable();

    return NULL;
}

// 获取数据量
int demo_frame_media_get_packet_num(void)
{
```

```
    struct demo_frame_media_rx_bulk *p;
    u32 num = 0;
    local_irq_disable();
    list_for_each_entry(p, &demo_frame_media_head, entry) {
        num++;
    }
    local_irq_enable();
    return num;
}



/* __attribute__((weak)) */
/* void demo_frame_media_rx_notice_to_decode(void) */
/* { */
/* } */

// 用 timer 模拟填数
static void demo_frame_test_time_func(void *param)
{
    struct demo_frame_media_rx_bulk *p;
    demo_frame_max += (DEMO_FRAME_TIME * SBC_DATA_SAMPRATE / 1000);
    /* printf("%s,%d \n", __func__, __LINE__); */
    /* printf("cnt:%d, max:%d \n", demo_frame_cnt, demo_frame_max); */
    while (demo_frame_cnt < demo_frame_max) {
        demo_frame_cnt += SBC_DATA_POINTS;
        // 申请空间
        p = lbuf_alloc((struct lbuff_head *)demo_frame_media_buf, sizeof(*p) +
sizeof(sbc_data));
        if (p == NULL) {
            /* putchar('!'); */
            continue ;
        }
```

```
        // 填数
        p->data_len = sizeof(sbc_data);
        memcpy(p->data, sbc_data, sizeof(sbc_data));
        local_irq_disable();
        list_add_tail(&p->entry, &demo_frame_media_head);
        local_irq_enable();
        // 告诉上层有数据
        demo_frame_media_rx_notice_to_decode();
    }
    if (demo_frame_cnt == demo_frame_max) {
        demo_frame_cnt = demo_frame_max = 0;
    }
}

// 模拟定时关闭
static void demo_frame_test_close(void *param)
{
    y_printf("%s,%d \n", __func__, __LINE__);
    sys_timer_del(demo_frame_test_tmr);
    demo_frame_test_tmr = 0;

    demo_frame_dec_close();

    local_irq_disable();
    __list_del_entry(&demo_frame_media_head);
    free(demo_frame_media_buf);
    demo_frame_media_buf = NULL;
    local_irq_enable();
}

void demo_frame_test(void)
{
```

```
    printf("%s,%d \n", __func__, __LINE__);

    // 申请空间
    u32 buf_size = 2 * 1024;
    void *buf = malloc(buf_size);
    ASSERT(buf);
    // 初始化 lbuf
    local_irq_disable();
    demo_frame_media_buf = buf;
    lbuf_init(demo_frame_media_buf, buf_size, 4, 0);
    local_irq_enable();

    // 用 timer 模拟填数
    demo_frame_test_tmr = sys_timer_add(NULL, demo_frame_test_time_func,
DEMO_FRAME_TIME);
    y_printf("id:%d \n", demo_frame_test_tmr);
    // 模拟定时关闭
    sys_timeout_add(NULL, demo_frame_test_close, 10 * 1000);

    // 启动解码
    demo_frame_dec_open();
}
```